

Lesson 3

Component Lifecycle, Hooks, State & Props

Code the Dream — React

Today's Session

EXPLORE

- Why regular variables can't drive a React UI
- Component lifecycle: mount, update, unmount
- Intro to hooks and the rules of hooks
- `useState`: storing and updating values
- Immutability and batched state updates
- Props: passing data from parent to child

APPLY

- Read buggy components and identify the issue
- Fix three bugs tied to this week's concepts
- Render all three in `studentWork.jsx`

By the end: understand why React needs state, use `useState` to store data, and pass that data to child components with props.

Why Regular Variables Don't Work

Changing a variable does **not** tell React to update the screen.

```
let message = 'Coming Soon...';

setTimeout(() => {
  message = 'We can feel it...'; // variable updates...
  console.log(message);          // ...but the page doesn't change
}, 3000);
```

React only re-renders a component when **state** changes. A plain variable update is invisible to React — it has no way to know the value changed.

This is the core problem that `useState` solves.

Component Lifecycle

Every React component goes through three phases:

- **Mount** — the component appears on the page for the first time
- **Update** — state or props change, triggering a re-render
- **Unmount** — the component is removed from the page

React automatically handles moving between these phases. Your job is to understand what triggers each one so you can write components that behave predictably.

State changes trigger **updates**. This is why using `useState` — not plain variables — makes the UI respond to data changes.

Intro to Hooks

Hooks are special functions that give components access to React's internal systems — like state, lifecycle, and refs.

Rules (always follow these):

- Call hooks at the **top level** of your component — not inside `if` statements, loops, or nested functions
- Only call hooks from **React function components** or custom hooks
- Custom hooks must start with "use" — e.g. `useCustomHook`

React relies on hook call order to pair each hook with its state. Breaking the rules causes subtle, hard-to-debug problems — ESLint's react-hooks plugin enforces them automatically.

The Three Hooks in This Course

Hook	Purpose
<code>useState</code>	Store a value and trigger re-renders when it changes
<code>useEffect</code>	Run code in response to lifecycle events
<code>useRef</code>	Hold a value across renders without triggering re-renders

This lesson focuses on `useState`. `useEffect` is introduced briefly in the exercise — its full API is covered in Lesson 8. `useRef` is covered in Lesson 4.

useState — Syntax

```
const [value, setValue] = useState(initialValue);
```

- `useState(initialValue)` — sets the starting value; can be any type
- `value` — the current state (read-only — never mutate it directly)
- `setValue` — the updater function; calling it triggers a re-render

Import it first:

```
import { useState } from 'react';
```

Convention: name the updater `set` + the variable name — e.g. `count / setCount`, `inventory / setInventory`.

useState — In Practice

```
import { useState } from 'react';
import inventoryData from './assets/inventory.json';

function App() {
  const [inventory, setInventory] = useState(inventoryData.inventory);

  return (
    <ul>
      {inventory.map((item) => (
        <li key={item.id}>
          <h2>{item.baseName}</h2>
          <p>{item.baseDescription}</p>
        </li>
      ))}
    </ul>
  );
}
```

Immutability — Don't Mutate State

React expects you to **replace** state values, not change them in place.

Wrong — mutates state directly

```
// Arrays
inventory.push(newItem);
setInventory(inventory);

// Objects
user.name = 'Alex';
setUser(user);
```

Correct — creates a new value

```
// Arrays
setInventory([...inventory, newItem]);

// Objects
setUser({ ...user, name: 'Alex' });
```

If you mutate state directly, React may not detect the change and the UI won't update. Always produce a new value.

Batched Updates & Functional Updating

When new state depends on old state, pass a **function** to the updater instead of a value.

Problem — stale value

```
function add() {  
  setCount(count + 1);  
  setCount(count + 1);  
  // Both reads see the same  
  // starting value → adds 1, not 2  
}
```

Fix — functional update

```
function add() {  
  setCount(prev => prev + 1);  
  setCount(prev => prev + 1);  
  // Each call gets the freshest  
  // value → adds 2  
}
```

React batches multiple `setState` calls for efficiency. The functional form guarantees each call sees the latest value.

Check for Understanding

What is wrong with this code?

```
const [items, setItems] = useState(['a', 'b', 'c']);

function addItem() {
  items.push('d');
  setItems(items);
}
```

- A) `useState` cannot hold an array
- B) State is being mutated directly before calling `setItems`
- C) `setItems` should be called without any argument
- D) There is nothing wrong — this is correct

Props — Passing Data Down

Props pass data from a parent component to its children. They flow in one direction: **down** the component tree.

Parent — owns the state, passes it down

```
function App() {  
  const [inventory, setInventory] =  
    useState(inventoryData.inventory);  
  return (  
    <ProductList inventory={inventory} />  
  );  
}
```

Child — receives and displays it

```
function ProductList({ inventory }) {  
  return (  
    <ul>  
      {inventory.map(item => (  
        <li key={item.id}>{item.baseName}</li>  
      ))}  
    </ul>  
  );  
}
```

Props are **read-only** in the child — only the parent that owns the state can update it.

Props Destructuring

Destructuring props in the function signature makes code cleaner and shows exactly what a component expects.

Without destructuring

```
function ProductCard(props) {  
  return (  
    <li>  
      <h2>{props.baseName}</h2>  
      <p>{props.baseDescription}</p>  
    </li>  
  );  
}
```

With destructuring

```
function ProductCard({  
  baseName,  
  baseDescription,  
}) {  
  return (  
    <li>  
      <h2>{baseName}</h2>  
      <p>{baseDescription}</p>  
    </li>  
  );  
}
```

You can also set **default values**: `function ProductCard({ baseName = 'Unknown' })`

The children Prop

Anything placed **between** a component's opening and closing tags becomes its `children` prop.

```
// Parent – nests a FeaturedItem inside ProductList
<ProductList inventory={inventory}>
  <FeaturedItem name="Limited Edition Tee" />
</ProductList>

// Child – renders {children} wherever it should appear
function ProductList({ inventory, children }) {
  return (
    <ul>
      {children}
      {inventory.map(item => (
        <ProductCard key={item.id} {...item} />
      ))}
    </ul>
  );
}
```

Check for Understanding

A child component receives a prop called `count`. The user clicks a button inside the child. What is the correct way to update `count`?

- A) Mutate the prop directly: `count = count + 1`
- B) Call `useState` inside the child with the prop as initial value and update from there
- C) The parent passes down the state setter as a prop; the child calls it
- D) Props can't be numbers — only strings and objects

Now: Apply

Three buggy components. One concept each. Let's fix them.

This Week's Exercise — React Bug Hunt

You have three broken components in `src/exercises/lesson-03/`. Each one has a bug tied to this week's concepts.

File	Bug
<code>BugEffectLoop.jsx</code>	Effect runs on every render instead of once
<code>BugMutatedState.jsx</code>	State is mutated directly
<code>BugProps.jsx</code>	Regular variable used instead of state

For each bug:

1. Read the code and identify what's wrong before making any changes
2. Fix the bug
3. Write a short explanation in the comment section at the bottom of the file

Bug #1 — Effect Runs on Every Render

Buggy

```
useEffect(() => {  
  setCount(count + 1);  
});  
// No dependency array →  
// runs after every render,  
// causing an infinite loop
```

Fixed

```
useEffect(() => {  
  setCount(count + 1);  
}, []);  
// Empty [] → runs once  
// after the first render  
// (mount) only
```

`useEffect` without a dependency array runs after **every** render. An empty `[]` tells React to run the effect only on **mount**.

Bug #2 — State Mutated Directly

Buggy

```
function handleAdd() {  
  count++;          // mutates state!  
  setCount(count);  
}
```

Fixed

```
function handleAdd() {  
  setCount(prev => prev + 1);  
}
```

`count++` mutates the state variable in place before passing it to `setCount`. React may not detect the change. Always derive the new value **inside** the setter — never touch the state variable directly.

Bug #3 — Regular Variable Instead of State

Buggy

```
function BugProps({ name }) {  
  let message = 'Hello, ' + name;  
  
  function handleChange() {  
    message = 'Hi, ' + name + '!';  
  }  
  return <p>{message}</p>;  
}
```

Fixed

```
function BugProps({ name }) {  
  const [message, setMessage] =  
    useState('Hello, ' + name);  
  
  function handleChange() {  
    setMessage('Hi, ' + name + '!');  
  }  
  return <p>{message}</p>;  
}
```

A `let` variable changes silently — React has no idea. Move the value into `useState` so React can track it and re-render when it changes.

Assemble in studentWork.jsx

Once all three bugs are fixed, render them together in `studentWork.jsx`:

```
import BugEffectLoop from './BugEffectLoop';
import BugMutatedState from './BugMutatedState';
import BugProps from './BugProps';
export default function StudentWork() {
  return (
    <div>
      <BugEffectLoop />
      <BugMutatedState />
      <BugProps name="Alex" />
    </div>
  );
}
```

Wrap-Up

This week you learned:

- Regular variables don't trigger re-renders — `useState` is how React tracks changing data
- Component lifecycle: mount → update → unmount
- Hooks must be called at the top level of a component, every render
- Props pass data **down** the tree; children can't update props directly
- Always replace state with a new value — never mutate it in place

Before next session: Complete the weekly assignment — add `useState` to your todo app, pass the list to `TodoList` via props, and create a `TodoListItem` component. Submit your PR link in CTD Learns. Post questions in **`#[class-name]-discussion`** on Slack.